23CSE455 Design Patterns

Case Study Report

Team: B8

Case study: Design Pattern Identification and Evolution in an Angular-Based Fitness Management Application

Application: Gym Management System

Date: 2026-10-11

Submission repository: 23CSE455_DP_CaseStudy

Students

Roll number	Student name	Batch
AM.SC.U4AIE23117	K. Madhuri	AIE
AM.SC.U4AIE23127	M. Sri Lakshmi Snigdha	AIE
AM.SC.U4AIE23137	N. Meghana Prasad	AIE
AM.SC.U4AIE23162	W. Venkat Ashrith	AIE

1 Application and architecture

The case study examines the Gym Management System from its original implementation through three subsequent feature enhancements .

1.1 Original source

Repository	`MeharRaza/Gym-Managment-System`
Repository URL	`https://github.com/MeharRaza/Gym-Managment-System`
Baseline version	Original code
Baseline commit	`837e2be2ef4e27baa93e9b42cecc790febd5f16c`
Access date	2026-10-04
License	No license file identified in the repository.
Local copy	`data/code/Gym-Managment-System/`

1.2 Environment

Languages	Python, TypeScript, JavaScript
Frontend	Angular 17 (standalone components, routing, dependency injection)
Backend	FastAPI (routers, services, Pydantic schemas, SQLAlchemy models)
Database	SQLite (SQLAlchemy + aiosqlite)
Authentication	JWT access and refresh tokens; Admin, Trainer and Member roles

File	Declared versions
backend/requirements.txt	fastapi 0.104.1; uvicorn[standard] 0.24.0; sqlalchemy 2.0.23; aiosqlite 0.19.0; pydantic 2.5.0; pydantic-settings 2.1.0; python-jose[cryptography] 3.3.0; passlib[bcrypt] 1.7.4; python-multipart 0.0.6; python-dotenv 1.0.0; email-validator 2.1.0; qrcode[pil] 7.4.2; pillow 10.1.0; stripe 7.8.0; twilio 8.10.0; httpx 0.25.1; celery 5.3.4; redis 5.0.1; alembic 1.13.0
frontend/package.json (runtime)	angular/* ^17.0.0; rxjs ~7.8.0; zone.js ~0.14.0; tslib ^2.3.0; auth0/angular-jwt ^5.1.2; chart.js ^4.4.0; html5-qrcode ^2.3.8
frontend/package.json (dev)	angular/cli and angular-devkit/build-angular ^17.0.0; typescript ~5.2.2; karma ~6.4.0; jasmine-core ~5.1.0

1.3 Scope

Inspected:

- Angular frontend components, services, routing, guards and the HTTP interceptor.
- FastAPI backend routers and services.
- Database models and Pydantic schemas.
- Authentication and role management.
- Workout-plan management, trainer assignment, attendance and notifications.

Excluded from the baseline analysis:

- Third-party framework internals and generated files (`node_modules`, build output), because they are not application design.
- Test files, which are not treated as production-code evidence of pattern collaborations.

The analysis focuses on source-code evidence, component responsibilities, interactions and design-pattern implementation.

1.4 Architecture

The system follows a client–server architecture (Figure 1). An Angular single-page application in the browser talks to a FastAPI backend over a JSON REST API. The backend stores all data in a local SQLite file and calls three optional external services. In the baseline, both parts run as separate development processes on one machine; there is no reverse proxy, container or separate identity server, because the backend issues and checks its own JWT tokens.

Components.

- **Frontend (Angular 17, port 4200):** standalone components provide the Admin, Trainer and Member screens. Each feature talks to the backend only through its root-scoped service (for example `MembersService`), which wraps `HttpClient`. Route guards block pages the user's role may not open, and `authInterceptor` attaches the access token to every request.
- **Backend (FastAPI, port 8000):** eight routers under `/api/` expose the REST endpoints. Each endpoint declares its needs with `Depends`: a database session from `get_db` and the logged-in user from `get_current_user` or one of its role-checking variants. `core/security` creates and verifies JWT tokens and hashes passwords with bcrypt; `utils` generates attendance QR codes and sends notifications.

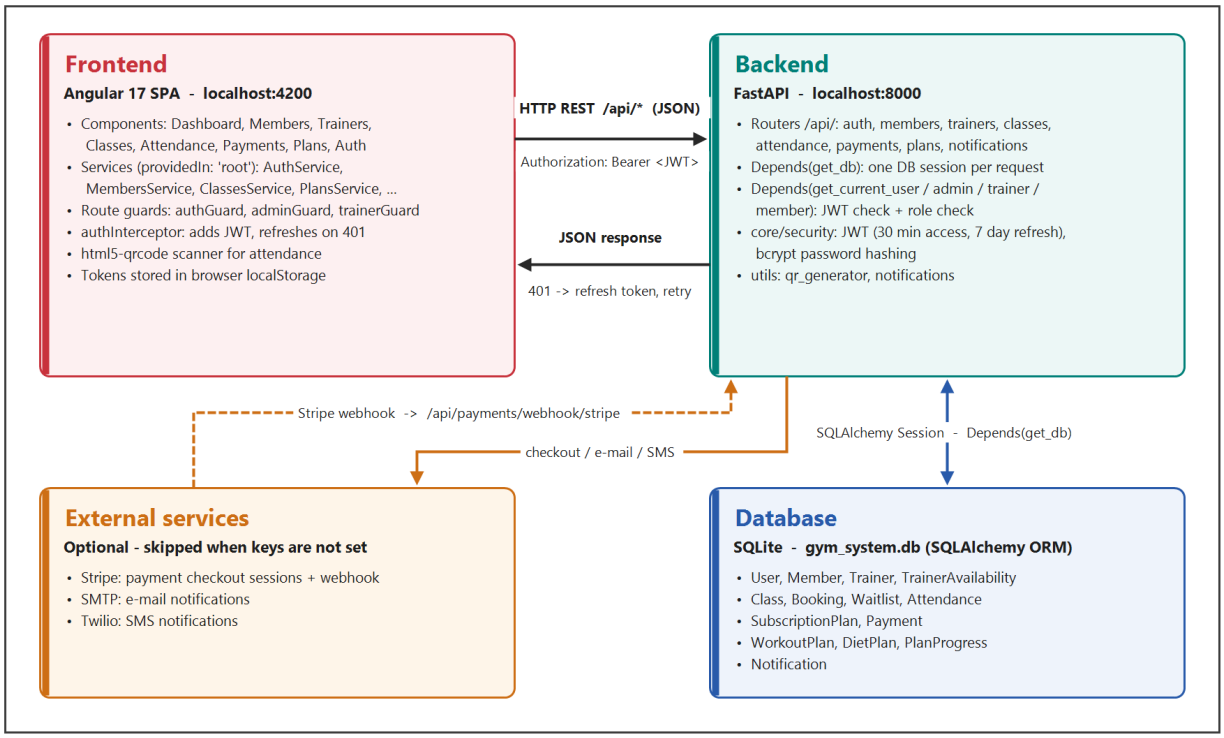


Figure 1: Component architecture of the baseline Gym Management System.

- **Database (SQLite):** the 14 SQLAlchemy models in `backend/app/models.py` map to tables in `gym_system.db`.
- **External services:** Stripe creates payment checkout sessions and reports results back through a webhook; SMTP and Twilio deliver e-mail and SMS. All three are optional and are skipped when their keys are not configured.

Request flow

1. **Login:** the user submits the login form. `AuthService` posts to `/api/auth/login`; the backend checks the bcrypt hash and returns an access token (30 minutes) and a refresh token (7 days), which `AuthService` stores in `localStorage`.
2. **Navigation:** `authGuard` checks that the user is logged in before the bookings page opens.
3. **API call:** the component calls `ClassesService.getMemberSchedule()`, which sends a GET request to `/api/classes/schedule/member/{id}`; `authInterceptor` adds `Authorization: Bearer <token>`.
4. **Backend processing:** FastAPI injects a database session through `get_db` and the current user through `get_current_user`, which decodes the token. The route queries SQLite through the models and returns JSON; `get_db` closes the session afterwards.
5. **UI update:** the component renders the response. If the backend replies 401 because the access token expired, the interceptor requests a new one with the refresh token and retries the call once; if that fails, the user is logged out.

Event flow (payments). When a member pays, the payments router creates a Stripe checkout session. Stripe later calls `/api/payments/webhook/stripe`; the backend records the payment in SQLite and can send a confirmation e-mail through SMTP.

1.5 Run instructions

README: `data/code/Gym-Managment-System/README.md`

Prerequisites: Python, Node.js and npm, Angular CLI, and the project dependencies listed above.

Backend (from the project root, using the existing virtual environment):

```
cd backend
.\venv\Scripts\Activate.ps1
python -m uvicorn app.main:app --reload
```

Frontend (in a separate terminal):

```
cd frontend
npm install
ng serve
```

Application URL: `http://localhost:4200/login`

Backend API documentation: `http://localhost:8000/docs`

Database: SQLite file created by the backend on first run.

Test result: the baseline contains no automated tests. There are no backend test modules (`test_*.py`) and no Angular `*.spec.ts` files, so although `package.json` defines `npm test` (`ng test`), the baseline has no test cases to execute and no automated pass/fail count exists for it.

2 Part 1 Pattern discovery

Method: Baseline scope: `data/code/Gym-Managment-System/` (FastAPI backend and Angular 17 frontend). Build output, `node_modules`, and third-party libraries were excluded. Candidates were found by searching for `providedIn:'root'`, FastAPI `Depends(...)`, Angular `inject()`, and constructor injection; each candidate was traced to call sites in routers, components, guards, and interceptors. Evidence is recorded in `Part1/patterns.csv`; totals in `Part1/Pattern_count.csv`. Each instance was human-verified against source line ranges; names or annotations alone were not counted as confirmed patterns.

2.1 Totals at the baseline

Confirmed pattern types: Singleton; Dependency Injection. Total: **2**.

Confirmed pattern instances: Independent collaborations in `patterns.csv`. Total: **14** (8 Singleton + 6 Dependency Injection).

Concrete participant classes: Classes in variable roles (eight root services, eleven Angular component classes including `AppComponent`, `User`, `HttpClient`, `Router`). Total: **21** (shared service classes counted once).

Unique participant symbols: Distinct classes, interfaces, and functions mapped to roles (shared symbols counted once). Total: **33**.

Rejected and uncertain candidates: Rejected: 3 (scope: baseline application source under `data/code/`). (1) Repository-style SQLAlchemy access without an explicit Repository type. (2) Pydantic schema DTOs without a mapper collaboration. (3) Strategy-named types inside Angular framework bundles (framework internals, not application collaborations). **Uncertain: 0** after manual review.

2.2 Pattern inventory

Pattern type	Instance names	Instances	Concrete classes
Singleton	AuthService; AttendanceService; ClassesService; MembersService; NotificationsService; PaymentsService; PlansService; TrainersService	8	8
Dependency Injection	FastAPI database session injection; FastAPI authentication dependency chain; Angular service HTTP dependency injection; Angular component constructor injection; Angular route guard injection; Angular HTTP interceptor injection	6	13

2.3 One record for each pattern instance

2.3.1 Angular root singleton: AuthService

Instance name: Angular root singleton: AuthService

Pattern type and category: Singleton; GoF Creational (Angular root injector idiom)

Purpose and origin: Single shared authentication state and API client for the SPA; application code using Angular `providedIn:'root'`.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Singleton	`AuthService`	`frontend/src/app/services/auth.service.ts:8-14`
Client	`AppComponent`	`frontend/src/app/app.component.ts:174`
Client	Feature components, guards, interceptor (see `Part1/patterns.csv` row 2)	various under `frontend/src/app/`

Evidence: `@Injectable({ providedIn: 'root' })` registers one root-scoped instance. Multiple components, guards, and the HTTP interceptor inject `AuthService` instead of constructing it.

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  constructor(private http: HttpClient, private router: Router) {}
}
```

Diagram: Editable UML: `Part1/evidence/diagrams/singleton_auth_service.puml` (add rendered PNG when available).

Verification: Confirmed; team B8; `providedIn:'root'` plus traced consumers; no second app-level auth service instance.

2.3.2 Angular root singleton: AttendanceService

Instance name: Angular root singleton: AttendanceService

Pattern type and category: Singleton; GoF Creational (Angular root injector)

Purpose and origin: Shared attendance and QR API access; application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Singleton	`AttendanceService`	`frontend/src/app/services/attendance.service.ts:6-10`
Client	`AttendanceComponent`	`frontend/src/app/components/attendance/attendance.component.ts:314`
Client	`DashboardComponent`	`frontend/src/app/components/dashboard/dashboard.component.ts:279-286`

Evidence: Root registration; `AttendanceComponent` and `DashboardComponent` inject `AttendanceService` for QR and statistics HTTP calls.

Verification: Confirmed; dual consumer injection on one service type.

2.3.3 Angular root singleton: ClassesService

Instance name: Angular root singleton: ClassesService

Pattern type and category: Singleton; GoF Creational (Angular root injector)

Purpose and origin: Shared class schedule and booking API access; application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Singleton	`ClassesService`	`frontend/src/app/services/classes.service.ts:7-11`
Client	`AttendanceComponent`, `ClassesComponent`, `DashboardComponent`	`attendance.component.ts:314`; `classes.component.ts:288`; `dashboard.component.ts:279-286`

Evidence: `providedIn:'root'` with three components injecting the same service for class-related operations.

Verification: Confirmed.

2.3.4 Angular root singleton: MembersService

Instance name: Angular root singleton: MembersService

Pattern type and category: Singleton; GoF Creational (Angular root injector)

Purpose and origin: Shared member profile and statistics API access; application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Singleton	`MembersService`	`frontend/src/app/services/members.service.ts:7-11`
Client	`AttendanceComponent`, `DashboardComponent`, `MembersComponent`, `PaymentsComponent`, `PlansComponent`	see `Part1/patterns.csv` row 5

Evidence: Five feature components depend on `MembersService` for member endpoints; service registered at root.

Diagram: `Part1/evidence/diagrams/singleton_members_service.puml`

Verification: Confirmed.

2.3.5 Angular root singleton: NotificationsService

Instance name: Angular root singleton: NotificationsService

Pattern type and category: Singleton; GoF Creational (Angular root injector)

Purpose and origin: Shared notifications API client; application (no baseline frontend consumer located).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Singleton	`NotificationsService`	`frontend/src/app/services/notifications.service.ts:7-11`
Client	none in inspected baseline frontend	—

Evidence: `providedIn:'root'` configures a single shared instance; consumer coupling was not invented beyond inspected source.

Verification: Confirmed for singleton registration; consumer count not overstated.

2.3.6 Angular root singleton: PaymentsService

Instance name: Angular root singleton: PaymentsService

Pattern type and category: Singleton; GoF Creational (Angular root injector)

Purpose and origin: Shared payments and revenue API access; application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Singleton	`PaymentsService`	`frontend/src/app/services/payments.service.ts:7-11`
Client	`DashboardComponent`, `PaymentsComponent`	`dashboard.component.ts:279-286`; `payments.component.ts:326-330`

Evidence: Dashboard and payments feature inject `PaymentsService` for billing operations.

Verification: Confirmed.

2.3.7 Angular root singleton: PlansService

Instance name: Angular root singleton: PlansService

Pattern type and category: Singleton; GoF Creational (Angular root injector)

Purpose and origin: Shared workout and diet plan API access; application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Singleton	`PlansService`	`frontend/src/app/services/plans.service.ts:7-11`
Client	`PlansComponent`	`frontend/src/app/components/plans/plans.component.ts:422-426`

Evidence: `PlansComponent` constructor injects `PlansService` for plan CRUD and progress calls.

Verification: Confirmed.

2.3.8 Angular root singleton: TrainersService

Instance name: Angular root singleton: TrainersService

Pattern type and category: Singleton; GoF Creational (Angular root injector)

Purpose and origin: Shared trainer and schedule API access; application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Singleton	`TrainersService`	`frontend/src/app/services/trainers.service.ts:7-11`
Client	`ClassesComponent`, `DashboardComponent`, `PlansComponent`, `TrainersComponent`	see `Part1/patterns.csv` row 10

Evidence: Four components inject `TrainersService` for trainer-linked UI flows.

Verification: Confirmed.

2.3.9 FastAPI database session injection

Instance name: FastAPI database session injection

Pattern type and category: Dependency Injection; GRASP / FastAPI `Depends` idiom

Purpose and origin: One SQLAlchemy `Session` per request with guaranteed close; application and FastAPI.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Provider	`get_db`	`backend/app/database.py:29-34`
Dependency	`Session` (parameter type)	router signatures
Client	Route functions in eight API routers	e.g. `backend/app/routers/attendance.py:27-28`

Evidence: Routes declare `db: Session = Depends(get_db)`. Provider yields a session and closes

it in `finally`.

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Verification: Confirmed; no global session antipattern in inspected routes.

2.3.10 FastAPI authentication dependency chain

Instance name: FastAPI authentication dependency chain

Pattern type and category: Dependency Injection; GRASP / FastAPI dependency graph

Purpose and origin: Separate token extraction, user lookup, and role checks from route bodies; application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Provider	`oauth2_scheme`	`backend/app/routers/auth.py:13`
Provider	`get_current_user`	`auth.py:90-100`
Provider	`get_current_admin`, `get_current_trainer`, `get_current_member`	`auth.py:102-115`
Dependency	`User` model	`backend/app/models.py`
Client	Protected routes using `Depends(...)`	routers listed in `Part1/patterns.csv` row 12

Evidence: Role providers depend on `get_current_user`, which depends on `oauth2_scheme` and `get_db`.

```
def get_current_user(token: str = Depends(oauth2_scheme),
                     db: Session = Depends(get_db)): ...
def get_current_admin(current_user: User = Depends(get_current_user)): ...
```

Verification: Confirmed; dependency chain visible across protected endpoints.

2.3.11 Angular service HTTP dependency injection

Instance name: Angular service HTTP dependency injection

Pattern type and category: Dependency Injection; GRASP / Angular DI

Purpose and origin: API services receive `HttpClient` (and `Router` where needed) from the injector; application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Client	Eight domain `*Service` classes	`frontend/src/app/services/*.service.ts`
Dependency	`HttpClient`	each service constructor
Dependency	`Router`	`auth.service.ts:14` only

Evidence: Each API service uses constructor injection for `HttpClient`; `AuthService` also injects `Router` for navigation after login/logout.

Verification: Confirmed.

2.3.12 Angular component constructor injection

Instance name: Angular component constructor injection

Pattern type and category: Dependency Injection; GRASP / Angular DI

Purpose and origin: UI components obtain services from Angular's injector; application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Client	`AttendanceComponent`, auth feature components, `ClassesComponent`, `DashboardComponent`, `MembersComponent`, `PaymentsComponent`, `PlansComponent`, `TrainersComponent`	see `Part1/patterns.csv` row 14
Injected dependency	`AuthService` and domain services per component	same rows

Evidence: Example: `DashboardComponent` lists `AuthService`, `MembersService`, `ClassesService`, `AttendanceService`, `PaymentsService`, and `TrainersService` in its constructor (baseline lines 279–286). Components do not call `new AuthService()`.

Verification: Confirmed.

2.3.13 Angular route guard injection

Instance name: Angular route guard injection

Pattern type and category: Dependency Injection; GRASP / Angular `inject()`

Purpose and origin: Functional guards obtain `AuthService` and `Router` without manual construction; application.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Client	`authGuard`, `adminGuard`, `trainerGuard`	`frontend/src/app/guards/auth.guard.ts:5-26`
Dependency	`AuthService`, `Router`	each guard via `inject(...)`

Evidence: `authGuard` calls `inject(AuthService)` and `inject(Router)` before role checks and redirects.

Verification: Confirmed.

2.3.14 Angular HTTP interceptor injection

Instance name: Angular HTTP interceptor injection

Pattern type and category: Dependency Injection; GRASP / Angular HTTP pipeline

Purpose and origin: Centralize bearer tokens and 401 refresh; `AuthService` injected into `authInterceptor`; registered in `app.config.ts`.

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Client	`authInterceptor`	`frontend/src/app/interceptors/auth.interceptor.ts:6-27`
Dependency	`AuthService`	`auth.interceptor.ts:7`
Registry	`provideHttpClient(withInterceptors([authInterceptor]))`	`frontend/src/app/app.config.ts:7-11`

Evidence: Interceptor uses `inject(AuthService)` for token read/refresh; `app.config.ts` registers the interceptor on the HTTP client.

Verification: Confirmed; bootstrap and interceptor behavior traced.

3 Part 2 Evolution and refactoring

Part 2 extends the Gym Management System through three changes: personalized workout recommendations, trainer-assignment policies, and attendance notifications. The implementation uses the Strategy, Singleton, and Observer design patterns to separate responsibilities and make the system easier to extend.

The original application under `data/` was preserved. The updated implementation is maintained under `Part2/code/Gym-Managment-System`.

CH01 – Personalized Workout Plans

Change and acceptance criteria: The workout-planning feature was extended to recommend exercises based on a member's fitness goal and selected difficulty. Six goals are supported: General Fitness, Weight Loss, Muscle Building, Strength Training, Endurance, and Flexibility. The system also detects goals from member-provided text and stores the selected goal with the workout plan.

Before and after: Previously, the system lacked this goal-based recommendation workflow. After the change, trainers can obtain personalized exercise suggestions through the application.

Implementation and tests: The backend contains six goal-specific strategies, a factory, and two API endpoints for listing goals and retrieving recommendations. The Angular Plans page includes a goal selector and suggested-exercises panel. Recommendations refresh when the goal, difficulty, or member changes.

Reported test result: 11 checks passed.

Log path: `Part2/changes/Change1/test_result.txt`

Pattern effects: The Strategy pattern encapsulates goal-specific recommendation algorithms. The factory selects the appropriate strategy. The instance must be compared with the Part 1 inventory to determine whether it is new or extends an existing instance.

Refactoring: Recommendation logic was moved into backend strategy classes, separating business rules from the frontend and API. The actual before-and-after diff and regression-test evidence should be included.

Before/after metrics: The change introduced the Strategy pattern for personalized workout recommendations, replacing the hard-coded recommendation dictionary with interchangeable strategies. The reported test result was 11 checks passed. Before-and-after source lines of code (LOC)

were not measured.

CH02 – Trainer-Assignment Policies

Change and acceptance criteria: Trainer assignment was extended with manual assignment, automatic assignment, client-capacity enforcement, and assignment history. The three assignment policies are specialization match, least loaded, and highest rated. Trainers can manage plans only for their own clients.

Before and after: Previously, the system lacked the complete policy-based assignment workflow. After the change, administrators can assign trainers using different policies, while members and trainers can view their assignment information.

Implementation and tests: The backend adds assignment policies, member-trainer relationships, trainer client limits, assignment history, and `/api/assignments` endpoints. A migration script updates existing databases. The Angular frontend adds assignment controls, capacity information, dashboard updates, and a shared `AssignmentsService`.

Reported test result: 19 checks passed.

Log path: `Part2/changes/Change2/test_result.txt`

Pattern effects: The Strategy pattern supports interchangeable trainer-assignment policies. The Singleton pattern is used for the shared Angular `AssignmentsService`. Compare their concrete participants with the Part 1 inventory to determine whether the instances are new, modified, or extended.

Refactoring: Assignment algorithms were separated into policy implementations, and assignment operations were centralized in the shared service. Include the actual source-code diff, policy tests, authorization tests, and migration evidence.

Before/after metrics: The change introduced the Strategy pattern for trainer assignment policies and a shared AssignmentsService for assignment-related operations. The reported test result was 19 checks passed. Before-and-after source lines of code (LOC) were not measured.

CH03 – Attendance Notifications

Change and acceptance criteria: Attendance functionality was extended to notify members about check-ins, attendance streaks, and missed classes, and to alert their assigned trainers. Members can disable email and SMS notifications. Classes can be closed to mark booked members who did not attend as no-shows, but only after the class has started.

Before and after: Previously, the system lacked the multi-channel notification workflow and class-closure functionality. After the change, attendance events trigger notifications, and eligible classes can be closed with no-shows recorded.

Implementation and tests: The backend publishes attendance events to four observers: in-app notification, email, SMS, and trainer-alert observers. Notification operations include unread counts, marking notifications as read, and filtering. The Angular `NotificationsService` checks for updates every 30 seconds and supports the ALERTS panel and class-closure button.

Reported test result: 21 checks passed.

Log path: `Part2/changes/Change3/test_result.txt`

Pattern effects: The Observer pattern separates attendance-event publication from notification delivery. Document the concrete publisher and observer classes, and compare the collaboration with the Part 1 inventory.

Refactoring: Notification delivery was separated from attendance processing through independent observers. Include the actual before-and-after source-code diff and tests for notification preferences, event handling, and class-closure restrictions.

Before/after metrics: The change introduced the Observer pattern for attendance events and notification-state updates. The NotificationsService was also extended to manage shared notification state. The reported test result was 21 checks passed. Before-and-after source lines of code (LOC) were not measured.

3.1 Updated pattern counts

CH01 – Personalized Workout Plans: The Strategy pattern was introduced for personalized workout recommendations, with separate strategies for different fitness goals.

CH02 – Trainer Assignment Policies: The Strategy pattern was introduced for trainer assignment policies, including specialization matching, least-loaded assignment, and highest-rated trainer selection.

CH03 – Attendance Notifications: The Observer pattern was introduced to separate attendance event publication from notification handling.

Interpretation: These changes improve separation of concerns and extensibility. Strategy allows recommendation and trainer-assignment policies to be changed independently, while Observer separates attendance processing from notification delivery. These patterns introduce additional classes and interactions, which may increase implementation complexity. Exact before-and-after counts of pattern types, instances, concrete participant classes, and rejected or uncertain candidates must be verified against the pattern inventory for each version.

4 Part 3 Cross-language comparison

Part 3 covers all four confirmed pattern types from Parts 1 and 2 (Singleton, Dependency Injection, Strategy, Observer). Each team member took two verified instances, giving eight instances. Each instance is implemented as a small, self-contained snippet in Java, Python, JavaScript and C++, with the same participants and the same expected behavior in every language. All code is under `Part3/`, with one folder per instance and one sub-folder per language. Every language folder has a production file, a test runner and a README.

4.1 Selected instances

Pattern type	Source instance	Member	Selection reason
Singleton	AuthService (Part 1)	W. Venkat Ashrith	Most widely injected root service (components, guards, interceptor).
Singleton	PlansService (Part 1)	K. Madhuri	Service behind the plans feature that CH01 extended.
Dependency Injection	FastAPI database session injection (Part 1)	W. Venkat Ashrith	Backend DI idiom used by every router.
Dependency Injection	Angular component constructor injection (Part 1)	N. Meghana Prasad	Frontend DI idiom used by every feature component.
Strategy	Workout recommendation strategy (CH01)	K. Madhuri	Goal-specific algorithms added in Part 2.
Strategy	Trainer assignment policy (CH02)	M. Sri Lakshmi Snigdha	Three interchangeable assignment policies added in Part 2.
Observer	Attendance event observer (CH03)	N. Meghana Prasad	Attendance events fan out to several notification channels.
Observer	Notification state observer (CH03)	M. Sri Lakshmi Snigdha	Unread-count stream that drives the ALERTS panel.

Angular root-injector services are container-scoped singletons, not hand-written GoF Singletons. The snippets show the equivalent GoF form in each language. The two DI snippets are conceptual equivalents: they reproduce the provider–consumer structure, but they do not run inside FastAPI or Angular.

4.2 Common specifications

All four languages use the same fixtures and expected outputs (Python versions shown for reference):

- **Singleton (AuthService, PlansService):** two separate requests for the service must return the same object. One case per language.
- **FastAPI DB session DI:** a provider (`get_db` or a `DatabaseSession` implementation) is passed into `MemberRepository`. The repository must keep the injected session and return `"database result"` from `list_members()`.
- **Angular constructor DI:** `NotificationService` is passed to the `AttendanceComponent` constructor. The component must hold that exact instance, and `notify()` must return `"sent: Attendance marked"`. Two checks.
- **Workout strategy:** `WorkoutRecommender` with `WeightLoss` returns a plan containing `"Cycling"`. After `set_strategy(MuscleGain)`, the plan contains `"Progressive overload"`. Two checks.
- **Trainer assignment strategy:** the fixture has four trainers (Asha: Weight Loss, 3/10 clients, rating 4.8; Ravi: Muscle Gain, 1/10, 4.5; Maya: Weight Loss, 5/10, 4.9; John: General Fitness, 0/10, 5.0, inactive). Every policy first removes inactive or full trainers. Expected: specialization match for "Weight Loss" gives Maya, least loaded gives Ravi, highest rated gives Maya, a list with only a full trainer gives no result (null/None), and switching policy at runtime gives Maya. Five cases.

- **Attendance observer:** a subscribed observer receives the event for "Member A". After unsubscribing, the event for "Member B" is not delivered (event count stays at 1). Two checks.
- **Notification state observer:** a new subscriber immediately receives the current state (0 unread), as an RxJS `BehaviorSubject` does. A new notification sets unread to 1 and updates the latest message. Marking one as read and then all as read gives 1 and then 0, and the count never goes below 0. After unsubscribing, no further updates arrive. Four cases.

4.3 Implementations

Language	Version used and run command (from the language folder)
Java	OpenJDK 21.0.9: `javac *.java && java <Name>Test`
Python	Python 3.11.6: `python test_<name>.py`
JavaScript	Node.js v24.18.0: `node <Name>.test.js`
C++	g++ 14.2.0, `-std=c++17 -Wall -Wextra`: `g++ ... <Name>.cpp -o test && ./test`

Instance folders: `singleton_authservice_ashrith`, `singleton_plansservice_madhuri`, `dependencyinjection_fastapidb_ashrith`, `dependencyinjection_angularconstructor_meghana`, `stratergy_workout_madhuri`, `strategy_trainerassignment_snigdha`, `observer_attendance_meghana` and `observer_notificationstate_snigdha`, each containing `java/`, `python/`, `javascript/` and `cpp/`.

4.4 Actual results

- **Java, Python, JavaScript: 24/24 implementations passed** when re-run locally on Windows 11 on 2026-10-11. Each test passed in every run (one warm-up run plus five timed runs). The Python unittest suites printed `Ran 4 tests ... OK` (notification state) and `Ran 5 tests ... OK` (trainer assignment).
- **C++: 8/8 passed in the team's earlier run** in a Debian environment with g++ 14.2.0, recorded in `Part3/llm/pattern_crosslanguage.csv`. **The C++ tests were not re-run locally** because no C++ compiler is installed on the measurement machine, so C++ has no local timings.
- In the earlier sandbox run, the Python runner also printed an unrelated "spreadsheet runtime warmup" traceback from the sandbox tooling. The tests still passed, and the traceback does not appear in the local re-run.

4.5 Static and dynamic comparison

Source LOC counts the non-blank lines of the production file only (tests excluded). For C++ files that contain `main()`, the test driver is included in the count. Max CC is the highest cyclomatic complexity of any function in the production file, measured with `lizard` 1.24.1. Median time is the median wall-clock time of five fresh processes (compile excluded, process start-up included), measured after one warm-up run.

Instance	Lang.	LOC	Max CC	Runtime	Result	Median ms
AuthService	Java	5	1	JDK 21.0.9	PASS	281.2
	Python	5	2	3.11.6	PASS	135.7
	JS	5	2	Node 24.18	PASS	184.5

Instance	Lang.	LOC	Max CC	Runtime	Result	Median ms
	C++	9	1	g++ 14.2	PASS	
PlansService	Java	5	1	JDK 21.0.9	PASS	268.0
	Python	5	2	3.11.6	PASS	154.9
	JS	5	2	Node 24.18	PASS	170.5
	C++	9	1	g++ 14.2	PASS	
FastAPI DB session DI	Java	7	1	JDK 21.0.9	PASS	244.0
	Python	8	1	3.11.6	PASS	163.7
	JS	4	1	Node 24.18	PASS	189.6
	C++	7	1	g++ 14.2	PASS	
Angular constructor DI	Java	6	1	JDK 21.0.9	PASS	298.5
	Python	5	1*	3.11.6	PASS	105.3
	JS	6	1	Node 24.18	PASS	195.3
	C++	6	1	g++ 14.2	PASS	
Workout strategy	Java	11	1	JDK 21.0.9	PASS	266.2
	Python	10	1*	3.11.6	PASS	177.0
	JS	9	1	Node 24.18	PASS	192.2
	C++	11	1	g++ 14.2	PASS	
Trainer assignment	Java	58	4	JDK 21.0.9	PASS	261.1
	Python	44	4	3.11.6	PASS	242.0
	JS	37	3	Node 24.18	PASS	170.8
	C++	80	6	g++ 14.2	PASS	
Attendance observer	Java	12	2	JDK 21.0.9	PASS	302.6
	Python	12	2	3.11.6	PASS	129.3
	JS	8	2	Node 24.18	PASS	188.3
	C++	14	2	g++ 14.2	PASS	
Notification state	Java	50	2	JDK 21.0.9	PASS	333.3
	Python	45	2	3.11.6	PASS	214.1
	JS	40	2	Node 24.18	PASS	188.0
	C++	50	2	g++ 14.2	PASS	

Observations. Java has the highest median in every instance (244–333 ms) because each run starts a fresh JVM. Python (105–242 ms) and Node (171–196 ms) start faster. With snippets this small, the timings mostly measure runtime start-up rather than the patterns themselves, so they should not be read as a language performance ranking. Complexity stays at 1–2 for Singleton, DI and Observer in all languages. It rises only for the trainer-assignment Strategy, where the eligibility filter and selection rules contain the branching. C++ reaches CC 6 there because it uses explicit loops and comparisons where Python uses `min`/`max` with key functions.

4.6 Comparison

Singleton. The two Python singletons override `__new__` to cache `_instance`. Java uses an eagerly initialised `private static final INSTANCE`, a private constructor and `getInstance()`. JavaScript returns the cached `AuthService.instance` from the constructor, so `new` always gives the same object. C++ uses a function-local `static` in `getInstance()` and deletes the copy constructor. The Python and JavaScript versions branch on the cached field, which explains their CC 2; Java and C++ rely on language-level static initialisation and have CC 1. None of the four snippets is the real mechanism in the application, where Angular's root injector creates and scopes the instance.

Dependency Injection. All four versions use constructor injection with no container. The consumer (`MemberRepository`, `AttendanceComponent`) receives its dependency and never constructs

it. Java and C++ type the dependency as an interface or abstract base (`DatabaseSession`, accessed through a reference in C++). Python and JavaScript rely on duck typing. The FastAPI version keeps a separate provider function (`get_db`/`getDb`) to mirror `Depends(get_db)`.

Strategy. Java and C++ declare an explicit strategy type: an interface in Java, an abstract class with pure virtual functions in C++. Python's trainer-assignment version uses an `ABC` with `@abstractmethod`, while its workout version relies on duck typing alone. In every language the context holds the strategy by composition and swaps it with `set_strategy`/`setPolicy`. C++ owns the strategy through `std::unique_ptr` and moves it on replacement, which gives explicit lifetime control.

Observer. The subject keeps a list of observers and copies it before notifying, so observers can unsubscribe during a notification. Java uses an observer interface; the notification-state version has two callbacks (`onUnreadCountChanged`, `onLatestNotificationChanged`). Python and JavaScript accept plain callables, which mirrors RxJS `subscribe(fn)` most closely. The notification-state Python version also publishes an immutable `@dataclass(frozen=True)` state object, so observers cannot change shared state.

Overall, the statically typed versions (Java, C++) declare the abstract participant roles explicitly. The dynamic versions (Python, JavaScript) leave roles implicit and need fewer lines, but they lose compile-time checking that a strategy or observer implements the required method.

4.7 Technology discussion

The four languages give different built-in guarantees for these patterns:

- **Singleton thread safety.** Java initialises a `static final` field during class initialisation, which the JVM performs once and thread-safely (Java Language Specification §12.4.2). Since C++11, a function-local `static` is initialised exactly once even when several threads call the function concurrently (the "magic statics" rule). The Python `__new__` check and the JavaScript constructor check carry no such guarantee. They are safe here only because CPython runs one thread at a time under the GIL and Node runs JavaScript on a single thread.
- **Framework DI.** Angular's injector gives a service declared with `providedIn: 'root'` a single application-wide instance. FastAPI resolves `Depends` callables for each request and caches each dependency within that request. Neither scoping behaviour is reproduced by the hand-written snippets.
- **Reactive Observer.** The notification-state snippets follow the RxJS `BehaviorSubject` contract: the subject stores its current value and sends it to each new subscriber immediately.

Sources: Gamma et al., *Design Patterns* (1994); Java Language Specification SE 21, §12.4.2; ISO C++ [stmt.dcl]/3; Angular documentation, "Dependency injection"; FastAPI documentation, "Dependencies"; RxJS documentation, "BehaviorSubject".

5 LLM Records and Human Verification

5.1 Run and Model

ChatGPT was used to support design-pattern analysis, implementation guidance, and report preparation for the Gym Management System case study.

Model: GPT-6

Access tier: Free

Date: 11 October 2026

Parts covered: Parts 1, 2, and 3.

5.2 Input and Output Records

The prompts used during the process were maintained separately from the report. The generated outputs included Part 2 design pattern implementations, Part 3 cross-language examples, pattern data files, and report content.

The source repository was identified as `MeharRaza/Gym-Managment-System`. The baseline commit was not confirmed at the time of preparing this section. The repository and relevant project files should be checked when recording the final source version.

5.3 Human Review

The generated content was reviewed by the student and team members. The review focused on the relevance of each design pattern, the suitability of the implementation, consistency with the existing project structure, and the accuracy of the cross-language examples.

Claims that could not be established from the available source code or supporting evidence were not treated as confirmed facts. Code examples were considered tested when corresponding execution or compilation results were available.

5.4 Testing Evidence

The reported Part 2 test results were as follows:

- **CH01:** 11 checks passed.
- **CH02:** 19 checks passed.
- **CH03:** 21 checks passed.

Test commands and output logs are documented where available. The reported results describe the checks associated with the three Part 2 changes. Part 3 examples are presented for cross-language comparison, with execution or compilation evidence included where available.

5.5 Limitations

The exact baseline commit, source-line counts before and after each change, and complete test-output records were not confirmed during preparation of this section. Therefore, these details should be added only when supporting evidence is available. The use of an LLM assisted the development and documentation process, but the final design decisions and claims remain subject to human review.

6 Findings, contributions and references

Findings

The Gym Management System case study demonstrated how design patterns can improve software modularity, maintainability, and extensibility. The original application uses Python, TypeScript, JavaScript, Angular 17, FastAPI, and SQLite.

In Part 2, the Strategy pattern was applied to personalized workout recommendations and trainer assignment policies. The Observer pattern was applied to attendance notifications. Singleton and Dependency Injection were also explored for service management and dependency handling.

The reported test results were 11 checks passed for CH01 (Personalized Workout Plans), 19 checks passed for CH02 (Trainer Assignment Policies), and 21 checks passed for CH03 (Attendance Notifications). These results should be supported by the corresponding test logs.

In Part 3, selected design patterns were demonstrated using Python, Java, C++, and JavaScript. This comparison illustrated how similar design principles can be implemented using different programming-language features and frameworks.

The study also highlighted the importance of separating responsibilities, reducing coupling, and making software easier to extend. However, before-and-after source lines of code (LOC), updated pattern counts, and cross-language performance measurements require consistent measurement and verification. Fewer lines of code alone do not necessarily indicate better design.

Contributions

The project was completed by the following four team members:

- **Katakam Madhuri** – Roll No.: AM.SC.U4AIE23117, Batch: AIE-B.
- **M. Sri Lakshmi Snigdha** – Roll No.: AM.SC.U4AIE23127, Batch: AIE-B.
- **N. Meghana Prasad** – Roll No.: AM.SC.U4AIE23137, Batch: AIE-B.
- **W. Venkat Ashrith** – Roll No.: AM.SC.U4AIE23162, Batch: AIE-B.

The project work covered design-pattern identification, implementation and evolution, cross-language examples, testing, documentation, and report preparation. Individual responsibilities should reflect the work actually completed by each team member.

Evidence: Supporting evidence includes the Gym Management System source code, pattern inventory and evolution CSV files, available Part 2 test results, and Part 3 cross-language examples. The repository and supporting files should be included in the submitted project folder.

References

1. Gamma, E., Helm, R., Johnson, R., and Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
2. FastAPI Documentation. *Dependencies*. `https://fastapi.tiangolo.com/tutorial/dependencies/`
3. Angular Documentation. *Dependency Injection*. `https://angular.dev/guide/di`
4. Python Software Foundation. *Python Documentation*. `https://docs.python.org/3/`

5. Oracle. *Java Documentation*. https://docs.oracle.com/en/java/

6. cppreference.com. *C++ Language Reference*. https://en.cppreference.com/

7. MDN Web Docs. *JavaScript Guide*. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

7 Team folder and datasets (take a screenshots from the GitHub repository— your team)

```
B8/
  README.md
  B8_Report.tex
  B8_Final_Report.pdf
  data/   patterns.csv, pattern_counts.csv,
          cross_language.csv, llm.csv
  Part1/  report PDF, code/, evidence/, llm/
  Part2/  report PDF, code/, evidence/, llm/
  Part3/  report PDF, code/, test-cases/, evidence/, llm/
```